

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0515 Slot B

COURSE OUTCOME COVERED

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)

Assessment Tool Weightage: 10%

QUESTIONS:4 Total Marks 20 Duration :60 Minutes Annexure A
Mock Interview

Code of Conduct:

I V.Thanusri(192472265) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: V.Thanusri

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge	
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem solving ability; requires guidance	Unable to solve or apply concepts	
Analytical Thinking	15	Analyzes questions critically and provides well structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated	
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively	
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism	

Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately	
-----------------------	----	--	---	------------------------------------	---------------------------------	--

OpenCV Basics & Pipeline Thinking

1. Design and explain a complete OpenCV pipeline to read, process, and display multiple images from a folder efficiently. Clearly justify your design choices and discuss potential challenges and optimizations.
2. A video file is not loading properly in OpenCV. Analyze the possible causes and propose a structured debugging approach, explaining each step logically.
3. You need to overlay text and shapes on a live video feed. Design your solution and explain how you would implement it efficiently, including key OpenCV functions and reasoning.
4. An image appears too dark when read. Analyze the issue and propose modifications to your processing pipeline, justifying how your approach improves visibility.

1. OpenCV Pipeline to Read, Process, and Display Multiple Images from a Folder

A suitable OpenCV pipeline is:

Folder → File Selection → Image Reading → Preprocessing → Processing → Display/Save

Steps

1. **Read image files:** Use `os.listdir()`, `glob`, or `pathlib` to obtain JPG, PNG, and other supported files.
2. **Load images:** Use `cv2.imread()` and check whether the image was successfully loaded.
3. **Preprocess:** Resize using `cv2.resize()`, convert to grayscale using `cv2.cvtColor()`, and reduce noise using `cv2.GaussianBlur()`.
4. **Process:** Apply the required operation, such as edge detection using `cv2.Canny()` or thresholding using `cv2.threshold()`.
5. **Display/save:** Use `cv2.imshow()` to display the original and processed image and `cv2.imwrite()` to save the output.

Pseudocode

BEGIN

 GET all image files from folder

 FOR each image file:

 image = `cv2.imread(file)`

```
    IF image is invalid:
        CONTINUE
    image = RESIZE(image)
    processed = PROCESS(image)
    DISPLAY processed image
    SAVE result if require
    WAIT for user input
    CLOSE all windows
END
```

Justification, Challenges and Optimization

Processing **one image at a time** reduces memory usage. Resizing large images improves processing speed. Invalid or corrupted images should be skipped instead of stopping the program. For very large folders, multiprocessing can be used to process independent images in parallel.

2. Debugging a Video File That Is Not Loading in OpenCV

A video may fail because of an incorrect path, corrupted file, unsupported codec, missing backend, or insufficient permissions.

Structured Debugging

1. **Check the file path:** Confirm that the file exists and the application has permission to read it.
2. **Open the video:**

```
cap = cv2.VideoCapture(video_path)
```

```
if not cap.isOpened():
```

```
    print("Video cannot be opened")
```

3. **Check video properties:** Verify FPS, width, height, and frame count using `cap.get()`.
4. **Test frame reading:**

```
ret, frame = cap.read()
```

```
if not ret:
```

```
print("Frame cannot be decoded")
```

5. **Check codec/backend:** If the file works in a media player but not OpenCV, the codec or OpenCV video backend may be the problem. Try another backend or convert the video to a commonly supported format.

Pseudocode

BEGIN

CHECK video file path

cap = OPEN video

IF video cannot be opened:

CHECK permissions

CHECK codec/backend

CHECK file corruption

EXIT

CHECK FPS, width and height

ret, frame = READ frame

IF ret == FALSE:

REPORT decoding error

RELEASE video

END

Conclusion

Debugging should proceed logically from **path** → **OpenCV opening** → **video properties** → **frame decoding** → **codec/backend**. This helps identify the exact cause instead of changing the program randomly.

3. Overlay Text and Shapes on a Live Video Feed

OpenCV can capture frames from a camera and draw graphical elements directly onto each frame.

Pipeline

Camera → Capture Frame → Process/Detect → Draw Shapes → Add Text → Display

Important OpenCV Functions

Function	Purpose
cv2.VideoCapture()	Capture live video
cv2.rectangle()	Draw rectangles
cv2.circle()	Draw circles
cv2.line()	Draw lines
cv2.putText()	Add text
cv2.imshow()	Display frame
cv2.waitKey()	Keyboard/loop control

Pseudocode

BEGIN

 camera = OPEN_CAMERA()

 IF camera cannot be opened:

 EXIT

 WHILE TRUE:

 ret, frame = READ_FRAME()

 IF ret == FALSE:

 BREAK

 DRAW rectangle on frame

 DRAW circle on frame

 DRAW line on frame

 ADD text using cv2.putText()

 DISPLAY frame

```
    IF user presses 'q':  
  
    BREAK  
  
    RELEASE camera  
  
    CLOSE windows  
  
END
```

Efficiency

Use a suitable camera resolution and avoid unnecessary copying of frames. Expensive operations such as object detection can be performed less frequently when appropriate. `cv2.waitKey(1)` keeps the video responsive.

Conclusion

OpenCV drawing functions provide a fast and simple way to add labels, bounding boxes, lines, and other graphics to live video while maintaining real-time performance.

4. Image Appears Too Dark When Read

An image may appear dark because of poor lighting, underexposure, low pixel intensity, or a narrow intensity range.

Proposed Processing Pipeline

Read Image → Analyze Intensity → Brightness/Contrast Correction → Enhancement → Display

Steps

1. Check image loading:

```
image = cv2.imread("image.jpg")
```

if image is None:

```
    print("Image loading failed")
```

2. Analyze intensity: Convert to grayscale using `cv2.cvtColor()` and check the histogram. If most pixels are near zero, the image is genuinely dark.

3. Adjust brightness and contrast: Use `cv2.convertScaleAbs()`.

```
enhanced = cv2.convertScaleAbs(
```

```
image,  
alpha=1.2,  
beta=30  
)
```

4. **Improve contrast:** cv2.normalize() or cv2.equalizeHist() can expand the intensity range.
5. **Handle uneven illumination:** CLAHE can be used:

```
clahe = cv2.createCLAHE(  
clipLimit=2.0,  
tileGridSize=(8, 8)  
)
```

```
enhanced = clahe.apply(gray)
```

Justification

- **Brightness adjustment** makes the entire image lighter.
- **Contrast enhancement** separates dark and bright regions.
- **Histogram equalization** improves global contrast.
- **CLAHE** improves local contrast and is useful for uneven lighting.

Conclusion

The image should first be analyzed rather than simply increasing brightness. For uniformly dark images, brightness and contrast adjustment are effective; for uneven illumination, **CLAHE** provides better visibility while limiting excessive enhancement and noise.